Três métodos de implantação

Nos módulos 1 e 2, você aprendeu a executar os agentes com `adk web`, a interface da web visual para desenvolvimento e teste. Você executou comandos como:

```
adk web
```

Esse comando abre uma interface baseada no navegador para você conversar com o agente de maneira interativa.

No entanto, o `adk web` não é o único meio de executar agentes. Neste módulo, você vai aprender outros três métodos:

- **`adk run`**: para execução com base em terminal
- **`adk api_server`**: para implantação como serviço de API
- **Execução programática**: para integrar agentes a aplicativos Python

Método 1: execução no terminal com `adk run`

Referência: [Documentação do ADK – Executar o agente] (em inglês)

O que é

O `adk run` permite que você interaja com o agente direto no terminal, sem abrir um navegador da web.

Como usar

No diretório do agente:

```
# Navegar até o diretório do projeto de agente
cd my_first_agent

# Executar o agente
adk run
```

Comportamento esperado:

- O terminal se torna interativo.
- Digite a mensagem e pressione **Enter**.
- O agente responde no terminal.
- Digite outra mensagem para continuar a conversa.
- Pressione `Ctrl+C` para sair.

No diretório pai:

```
# Se você estiver no diretório pai (por exemplo, adk-workspace)
adk run my_first_agent
```

Exemplo de interação

```
$ adk run my_first_agent

Você: Como  eu resolvo x + 5 = 10?
Agente: Excelente pergunta! Vamos resolver esse problema juntos. O que você acha que
precisamos fazer para encontrar apenas x em um único lado?

Você: Subtrair 5 dos dois lados?
Agente: Exatamente! Essa é a abordagem correta.  Quando nós subtraímos 5 dos dois
lados...
```

Quando usar `adk run`

Ideal para:

- ✅ Testes rápidos durante o desenvolvimento
- ✅ Fluxos de trabalho de linha de comando
- ✅ Ambientes de servidor sem GUI
- ✅ Scripts de teste automatizados
- ✅ Pipelines de CI/CD

Não é ideal para:

- ❌ Apresentar às partes interessadas (em vez dele, use `adk web`)
- ❌ Depurar conversas complexas (em vez dele, use `adk web`)

Método 2: servidor de API com `adk api_server`

Referência: [Documentação do ADK – Agent Config: executar]() (em inglês)

O que é

O `adk api_server` executa o agente como um serviço de API REST, permitindo que outros aplicativos enviem solicitações ao agente por HTTP.

Como usar

Inicie o servidor de API:

```
# No diretório do agente
cd my_first_agent
adk api_server
```

Ou no diretório pai:

```
adk api_server my_first_agent
```

Resultado esperado:

```
INFO:     Started server process [12345]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Como testar com cURL

Depois que o servidor estiver em execução, você poderá enviar solicitações usando `curl`:

```
# Em uma janela do terminal separada
curl -X POST http://localhost:8000/your-endpoint \
  -H "Content-Type: application/json" \
  -d '{"message": "Qual é o resultado de 2x + 5 = 13?"}'
```

Observação: para conferir informações detalhadas sobre o uso da API, consulte a documentação de teste do ADK (em inglês).

Quando usar o `adk api_server`

Ideal para:

- ✅ Integrar agentes a aplicativos da web
- ✅ Back-ends de apps para dispositivos móveis
- ✅ Arquiteturas de microsserviços
- ✅ Testes em pré-produção
- ✅ Desenvolvimento de API local antes da implantação no Cloud Run

Não é ideal para:

- ❌ Desenvolvimento interativo (em vez dele, use `adk web`)
- ❌ Testes rápidos (em vez dele, use `adk run`)

Método 3: execução programática com Python

Referência: Documentação do ADK – Tutorial da equipe de agentes (em inglês)

O que é

Você tem controle programático total para executar agentes diretamente no código Python. Ideal para:

- Criar aplicativos personalizados com agentes
- Notebooks Jupyter/Google Colab
- Pipelines de tratamento de dados
- Integrações personalizadas

Exemplo completo (pronto para copiar e colar)

Este é um exemplo independente e está pronto para ser executado em um script Python ou notebook Jupyter:

```
"""
Exemplo completo: executar um agente do ADK de modo programático
Copie este bloco de código inteiro para executá-lo em um script ou notebook Python.
"""

# Etapa 1: instalar o ADK (faça a execução no terminal ou na célula do notebook)
# pip install google-adk

# Etapa 2: definir a chave de API
# Opção A: definir como variável de ambiente antes da execução
#   export GOOGLE_API_KEY=your-api-key-here
# Opção B: remover a marca de comentário e usar este código:
# import os
# os.environ['GOOGLE_API_KEY'] = 'your-api-key-here'
# os.environ['GOOGLE_GENAI_USE_VERTEXAI'] = 'FALSE'

# Etapa 3: importar as bibliotecas necessárias
import asyncio
from google.adk.agents.llm_agent import Agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Etapa 4: definir o agente
agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor',
    instruction="""Você é um orientador de matemática paciente.
    Ajude os estudantes nas etapas de resolução dos problemas.
    Não dê as respostas de uma vez, ajude-os a encontrar as soluções."""
)

# Etapa 5: configurar a sessão e o Runner
APP_NAME = "math_tutor_app"
USER_ID = "student_1"
SESSION_ID = "session_001"

session_service = InMemorySessionService()
runner = Runner(
    agent=agent,
    app_name=APP_NAME,
    session_service=session_service
)
```

```python
# Etapa 6: definir a função assíncrona para executar o agente
async def run_agent():
    # Criar a sessão
    session = await session_service.create_session(
        app_name=APP_NAME,
        user_id=USER_ID,
        session_id=SESSION_ID
    )
    print(f"Session created: {SESSION_ID}\n")

    # Preparar a mensagem do usuário
    user_message = Content(
        role="user",
        parts=[Part(text="Como eu resolvo 2x + 5 = 13?")]
    )

    # Executar o agente e coletar a resposta
    print("Usuário: Como eu resolvo 2x + 5 = 13?\n")
    print("Agente: ", end="")

    async for event in runner.run_async(
        user_id=USER_ID,
        session_id=SESSION_ID,
        new_message=user_message
    ):
        # Mostrar a resposta final
        if event.is_final_response() and event.content and event.content.parts:
            print(event.content.parts[0].text)

# Etapa 7: executar o agente
# Para Jupyter/Colab: usar await diretamente
# await run_agent()

# Para scripts Python: usar asyncio.run()
asyncio.run(run_agent())
```

Como executar este exemplo

No notebook Jupyter ou no Google Colab:

```python
# Usar apenas await (o loop de eventos já está em execução)
await run_agent()
```

No script Python:

```python
# Usar asyncio.run() para iniciar o loop de eventos
asyncio.run(run_agent())
```

Resultado esperado:

```
Session created: session_001
```

```
Usuário: Como eu resolvo 2x + 5 = 13?

Agente: Excelente pergunta! Vamos resolver esse problema juntos. Primeiro o que você
acha que nós precisamos fazer para encontrar apenas x em um único lado?
```

Quando usar a execução programática

Ideal para:

- ✅ Notebooks Jupyter e Google Colab
- ✅ Aplicativos Python personalizados
- ✅ Pipelines de tratamento de dados
- ✅ Pesquisas e experimentos
- ✅ Controle minucioso da execução

Não é ideal para:

- ❌ Testes interativos rápidos (use `adk web` ou `adk run`)
- ❌ Ambientes que não são Python

Comparação: quando usar cada método

Método	Ideal para	Persistência da sessão	Complexidade da instalação
`adk web`	Desenvolvimento, depuração e demonstrações	✅ Sim (no navegador)	Baixa
`adk run`	Testes rápidos, fluxos de trabalho da CLI	❌ Não	Muito baixa
`adk api_server`	Integração de API, produção	Depende do cliente	Baixa
Programático	Apps personalizados, notebooks	✅ Sim (gerenciada)	Média

Resumo:

- **Desenvolvimento?** Use o `adk web`
- **Teste rápido?** Use o `adk run`
- **Criação de API?** Use o `adk api_server`
- **Integração personalizada?** Use a execução programática

Pontos principais

Escolha a ferramenta certa:

- Use o `adk web` para desenvolvimento e depuração visuais.
- Use o `adk run` para testes rápidos de linha de comando.
- Use o `adk api_server` para implantação como API.
- Use a execução programática para aplicativos Python personalizados.

Todos os métodos funcionam com o mesmo agente:

- O arquivo `agent.py` não muda.
- Outros métodos são apenas maneiras diferentes de interagir.
- Escolha de acordo com suas necessidades de fluxo de trabalho atuais.